

Game Input Bug Explanation

Generated: 2026-09-30T13:16:57.741Z • Source: Shared ChatGPT link

USER

```
game/game_engine.py import random import pygame from game.rope import Rope from game.player
import Puller
```

```
class GameEngine: def init(self, width, height): self.width = width self.height = height self.rope =
Rope(width, height) self.player = Puller(90, height // 2, (50, 120, 220), "PLAYER (A/D)") self.computer =
Puller(width - 90, height // 2, (220, 80, 50), "COMPUTER")
```

```
self.last_key = None self.is_pull_locked = False self.winner = None self.game_state = "PLAYING"
```

```
self.computer_pull_cooldown = 180 self.last_computer_pull = pygame.time.get_ticks()
```

```
self.font_big = pygame.font.SysFont(None, 48) self.font_small = pygame.font.SysFont(None, 26)
```

```
def handle_event(self, event): if self.game_state != "PLAYING": if event.type == pygame.KEYDOWN and
event.key == pygame.K_r: self.reset() return #BUG SYMPTOM:
```

Rapidly alternating between 'A' and 'D' causes the rope to suddenly

stop responding. The Computer easily pulls the rope to its side and wins while the player tries hard to pull the rope to its side.

```
if event.type == pygame.KEYDOWN: if event.key in (pygame.K_a, pygame.K_d): if not self.is_pull_locked:
if event.key != self.last_key: self.rope.pull_left(1.0) self.last_key = event.key self.is_pull_locked = True elif
event.type == pygame.KEYUP: if event.key == self.last_key: self.is_pull_locked = False
```

```
def update(self): if self.game_state != "PLAYING": return
```

```
now = pygame.time.get_ticks() if now - self.last_computer_pull >= self.computer_pull_cooldown:
computer_variance = random.uniform(0.7, 1.2) self.rope.pull_right(computer_variance)
self.last_computer_pull = now
```

RESULT

```
= self.rope.check_winner() if result: self.winner = result self.game_state = "GAME_OVER"
```

```
def reset(self): self.rope.reset() self.last_key = None self.is_pull_locked = False self.winner = None
self.game_state = "PLAYING" self.last_computer_pull = pygame.time.get_ticks()
```

```
def render(self, screen): screen.fill((30, 32, 36))
```

```
mud_rect = pygame.Rect(self.width // 2 - 120, self.height // 2 - 80, 240, 160) pygame.draw.rect(screen,
(45, 38, 30), mud_rect, border_radius=12)
```

```

self.rope.render(screen) self.player.render(screen) self.computer.render(screen)

inst_surf = self.font_small.render( "Alternate [A] and [D] keys rapidly to pull!", True, (210, 210, 210) )
screen.blit(inst_surf, (self.width // 2 - inst_surf.get_width() // 2, 40))

if self.game_state == "GAME_OVER": overlay = pygame.Surface((self.width, self.height),
pygame.SRCALPHA) overlay.fill((0, 0, 0, 180)) screen.blit(overlay, (0, 0))

win_text = f"{self.winner} WINS!" color = (80, 220, 80) if self.winner == "PLAYER" else (240, 80, 80)
text_surf = self.font_big.render(win_text, True, color) screen.blit( text_surf, (self.width // 2 -
text_surf.get_width() // 2, self.height // 2 - 50) )

restart_surf = self.font_small.render( "Press [R] to Play Again", True, (240, 240, 240) ) screen.blit(
restart_surf, (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 10) )game/rope.py import
pygame

class Rope: def init(self, screen_width, screen_height): self.screen_width = screen_width
self.screen_height = screen_height self.center_y = screen_height // 2 self.marker_x = screen_width // 2

self.left_win_x = 180 self.right_win_x = screen_width - 180 self.pull_step = 12

def pull_left(self, strength=1.0): self.marker_x -= int(self.pull_step * strength)

def pull_right(self, strength=1.0): self.marker_x += int(self.pull_step * strength)

def check_winner(self): if self.marker_x <= self.left_win_x: return "PLAYER" if self.marker_x >=
self.right_win_x: return "COMPUTER" return None

def reset(self): self.marker_x = float(self.screen_width // 2) self.velocity = 0.0

def render(self, surface): pygame.draw.line( surface, (180, 140, 90), (60, self.center_y), (self.screen_width
- 60, self.center_y), 10 )

pygame.draw.line( surface, (50, 200, 50), (self.left_win_x, self.center_y - 40), (self.left_win_x, self.center_y
+ 40), 4 ) pygame.draw.line( surface, (200, 50, 50), (self.right_win_x, self.center_y - 40), (self.right_win_x,
self.center_y + 40), 4 )

pygame.draw.line( surface, (120, 120, 120), (self.screen_width // 2, self.center_y - 20), (self.screen_width
// 2, self.center_y + 20), 2 )

flag_rect = pygame.Rect(int(self.marker_x) - 12, self.center_y - 24, 24, 48) pygame.draw.rect(surface,
(230, 40, 40), flag_rect, border_radius=4) pygame.draw.rect(surface, (255, 255, 255), flag_rect, width=2,
border_radius=4)game/player.py import pygame

class Puller: """Represents a puller character anchor on either side of the rope."""

def init(self, x, y, color, label): self.x = x self.y = y self.color = color self.label = label self.font =
pygame.font.SysFont(None, 24)

def render(self, surface): """Draw avatar and label."""

```

Body

```
body_rect = pygame.Rect(self.x - 20, self.y - 35, 40, 70) pygame.draw.rect(surface, self.color, body_rect, border_radius=6)
```

Head

```
pygame.draw.circle(surface, (240, 210, 180), (self.x, self.y - 50), 16)
```

Name / control tag

```
label_surf = self.font.render(self.label, True, (240, 240, 240)) surface.blit(label_surf, (self.x - label_surf.get_width() // 2, self.y + 45))main.py import pygame from game.game_engine import GameEngine
```

```
WIDTH, HEIGHT = 800, 450 FPS = 60
```

```
def main(): pygame.init() screen = pygame.display.set_mode((WIDTH, HEIGHT)) pygame.display.set_caption("Tug of War - Pygame Edition") clock = pygame.time.Clock()
```

```
engine = GameEngine(WIDTH, HEIGHT)
```

```
running = True while running: for event in pygame.event.get(): if event.type == pygame.QUIT: running = False engine.handle_event(event)
```

```
engine.update() engine.render(screen)
```

```
pygame.display.flip() clock.tick(FPS)
```

```
pygame.quit()
```

```
if name == "main": main()readme.md**# Tug of War Repair Lab**
```

This project is a competitive tug-of-war game using **Pygame**. It introduces students to alternating keyboard input synchronization, debounce and lock state patterns, physics-based tension balancing, and autonomous AI pulling mechanics within an object-oriented codebase.

What's Provided

A working Tug of War game with:

- A horizontal rope setup with goal markers, center boundary indicators, and a center position flag
- Two anchor pullers (`PLAYER` and `COMPUTER`) rendered with visual team colorings
- An alternating key input model requiring players to alternate `A` and `D` to pull left
- An autonomous computer opponent that pulls the rope rightward at recurring intervals
- Win-state boundary detection and a Game Over overlay with rematch support

It has **one deliberate bug** and **three optional features** left as tasks to implement. You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional and more interesting.

***** **Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.****

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

```
```bash
```

```
pip install pygame
```

```
```
```

3. Run the game:

```
```bash
```

```
python main.py
```

```
```
```

****Controls:**** Left-click on the HIGHER or LOWER buttons to predict the next card.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the input lock deadlock bug

When rapidly alternating between A and D, the rope suddenly locks up and stops responding to any player input, giving the computer a free win. In `game_engine.handle_event()`, `self.is_pull_locked` is set to `True` on key down, but it only unlocks on `KEYUP` if `event.key == self.last_key`. If the player presses the second key before fully releasing the first key (standard rapid keystroke overlap), `self.last_key` gets updated, preventing the previous key release from ever setting `self.is_pull_locked = False`. Remove the broken locking flag mechanism or redesign the debounce logic so simply alternating keys (`event.key != self.last_key`) successfully pulls without permanently freezing input.

Task 2: Implement dynamic computer difficulty surge

Right now, the computer pulls at a fixed interval of 180ms with minor random variance. Implement dynamic difficulty in `game_engine.update()`: if the center flag gets pulled closer to the player's goal line (`self.rope.left_win_x`), have the computer enter a "panic surge" mode by decreasing its cooldown or increasing its pulling strength to fight back aggressively

Task 3: Implement rope tension and pull animations

Currently, the rope is drawn as a static straight horizontal line, and puller positions do not react to movement. In `rope.render()` and `puller.render()`, implement rope sag or tension effects (e.g., slight

vertical vibration/waviness when tension is high) and add a leaning animation to each puller that tilts backwards according to who has pulling momentum

Task 4: Implement a match timer and sudden death mode

If two evenly matched opponents play, a match can last indefinitely. Add an active match timer displayed at the top of the screen. If neither side has won after 45 seconds, enter "Sudden Death" mode: double the pull distance of every keystroke and computer tick so the match finishes quickly.

Expected Behavior

- Rapidly alternating between A and D reliably moves the red center flag to the left without freezing or dropping inputs during fast mashing.
- The computer pulls the marker to the right at recurring intervals.
- The match ends when the flag crosses the left boundary (Player wins) or right boundary (Computer wins).
- Pressing R on the Game Over screen resets the rope marker, keys, timers, and game states.

Folder Structure

...

tug_of_war/

game/

game_engine.py

player.py

rope.py

main.py

README.md

...

Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay ****before**** your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay ****after**** your changes, showing the bug fixed and the new features working

- [] The Chat/LLM used page link, with the complete chat historyinstructionsLab 4: VibeCoding Objective Use Vibe Coding tools to fix the broken code and add features to the same. Learn how to prompt to get the job done in under three to four attempts. Clone a repo assigned to you, make changes and push to your repo in your personal account. THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!! Duration 60 minutes Software Requirements Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc) Instructions:

1. Go to the repo assigned to you
2. Go through the ReadMe file to understand the deliverables
3. Clone or Fork the repo
4. Run the Python based code (A game/ A Puzzle)
5. Record 10 seconds of video before making any changes.
6. Write prompts and fix the broken code
7. Write prompts to add all the features listed in the ReadMe file of the repo one by one.
8. Record another 10 seconds of video again after making all the changes.
9. Push the code to your repo. 10.Do not raise PR to the main repo (SETAPESU26) Deliverables: Upload the following to your Lab repo under Lab-4 folder, a. Videos (before and after) b. Updated Code c. Chat history exported as a doc/pdf----I'm working on a Pygame lab called "Tug of War Repair Lab". I've attached all the project files (main.py, game/game_engine.py, game/rope.py, game/player.py). The game has one deliberate bug and three features I need to implement. I want to do this one task at a time, and I want to understand every change, so for each task please:
10. Explain the root cause or your design idea first.
11. Show only the changed code (with the file name and the method it belongs in).
12. Tell me how to test it. Please don't rewrite everything at once. First, read the files and summarize in a few lines how the game currently works (input handling, computer pulling, win detection). Don't change any code yet.

ASSISTANT

Current game flow

- **Input:** GameEngine.handle_event() listens for A/D key presses. On a valid keydown, it calls rope.pull_left(1.0). It tries to enforce alternating keys using last_key plus is_pull_locked.
- **Computer:** GameEngine.update() checks every frame whether **180 ms** have passed since the last computer pull. If so, it calls rope.pull_right() with a random strength between 0.7 and 1.2.
- **Rope movement:** Rope stores the center flag position in marker_x. Player pulls decrease it; computer pulls increase it.
- **Win detection:** Rope.check_winner() returns "PLAYER" when marker_x <= left_win_x and "COMPUTER" when marker_x >= right_win_x. GameEngine.update() then changes the game state to GAME_OVER.
- **Reset:** Pressing R after game over resets the rope position and the relevant game state.

Task 1 bug — what is happening

The problem is the interaction between these two variables:

```
self.last_key
self.is_pull_locked
```

Suppose the player does:

A down D down A up

When D is pressed, last_key becomes D. Then when A is released, this condition fails:

```
if event.key == self.last_key:
```

because A != D.

So is_pull_locked stays True. If the next key events don't produce the exact state the code expects, player input can remain locked even though the player is still alternating keys.

I won't change anything yet. **Task 1 should be our first modification**, and after that we can test it before moving to the three optional features.

USER

Task 1: Fix the input lock deadlock bug. Symptom: when I rapidly alternate A and D, the rope suddenly stops responding to my input and the computer wins. In `GameEngine.handle_event()`, `self.is_pull_locked` is set to True on KEYDOWN, but it is only reset to False on KEYUP if `event.key == self.last_key`. When I press the second key before fully releasing the first one (overlapping keystrokes), last_key changes, so the first key's KEYUP never unlocks the lock. Please explain exactly why this deadlock happens, then fix it so that simply alternating keys (`event.key != self.last_key`) always pulls, with no permanent freeze even during fast mashing. Either remove `is_pull_locked` or redesign the debounce logic. Also make sure `reset()` stays consistent with your change.

ASSISTANT

Task 1 — Fix the input deadlock

Why the deadlock happens

The current logic uses **two pieces of state** to control one thing:

```
self.last_key
self.is_pull_locked
```

Consider this rapid sequence:

1. Press **A**

- last_key = A
- is_pull_locked = True

- Player pulls left.

2. Before releasing A, press **D**

- last_key = D
- is_pull_locked is already True
- Because of this condition:

```
if not self.is_pull_locked:
```

D doesn't pull at all.

3. Release **A**

- The code checks:

```
if event.key == self.last_key:
```

But now:

- released key = A
- last_key = D

So the condition is false.

- is_pull_locked remains True.

4. Now the player presses **D** or **A** again.

- not self.is_pull_locked is still False.
- Therefore, no more player pulls happen.

So the problem isn't really the alternating-key requirement. The problem is that **a KEYUP event from an older key is being used to unlock a lock whose state has already been associated with a newer key.**

Design change

We don't actually need is_pull_locked.

The README's desired behavior is simpler:

```
A key press should pull whenever it is different from the previous successful key press.
```

So we'll use only:

```
self.last_key
```

On every KEYDOWN:

- If it's A or D

- And it is different from last_key
- Pull left and update last_key

KEYUP no longer needs to do anything.

This also handles overlapping key presses correctly because the release order no longer matters.

Changed code

game/game_engine.py — handle_event()

Replace only the current handle_event() method with:

```
def handle_event(self, event):
    if self.game_state != "PLAYING":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if event.type == pygame.KEYDOWN:
        if event.key in (pygame.K_a, pygame.K_d):
            if event.key != self.last_key:
                self.ropes.pull_left(1.0)
                self.last_key = event.key
```

NOTICE

that we completely removed:

```
self.is_pull_locked
```

and the entire KEYUP section.

game/game_engine.py — __init__()

Since is_pull_locked is no longer used, remove this line:

```
self.is_pull_locked = False
```

Keep:

```
self.last_key = None
```

So that section becomes:

```
self.last_key = None
self.winner = None
self.game_state = "PLAYING"
```

game/game_engine.py — reset()

Remove:

```
self.is_pull_locked = False
```

So reset() becomes:

```
def reset(self):
    self.rope.reset()
    self.last_key = None
    self.winner = None
    self.game_state = "PLAYING"
    self.last_computer_pull = pygame.time.get_ticks()
```

This is important because after resetting, last_key = None means the **first A or D press is always accepted**.

How to test Task 1

Run:

```
python main.py
```

Test 1 — normal alternating input

Press:

A D A D A D

The red flag should consistently move left on every alternating key press.

Test 2 — rapid mashing

Try very quickly:

A D A D A D A D

The rope should **keep responding** instead of eventually freezing.

Test 3 — overlapping keys

Hold **A**, then press **D**, then release **A**, then release **D**.

Then start:

A D A D

The game should still accept the alternating presses. The release order should no longer be capable of creating a deadlock.

Test 4 — repeated same key

Press:

A A A A

Only the first A should pull.

Then:

D

should pull again because it is different from the previous successful key.

Test 5 — reset

Let the game finish, press **R**, then immediately press either **A** or **D**.

The first key should work because:

```
self.last_key = None
```

after reset.

One small point: with this fix, pressing the second key while the first is still physically held *will* count as a valid alternating press. That's exactly what the task asks for; KEYUP timing no longer controls whether the game becomes locked.

USER

Before I apply this, please review your own fix critically: what edge cases could still break it (for example holding a key down, key auto-repeat, pressing A and D at the same instant, pressing the same key twice)? Should the first press of the game work for either key?

ASSISTANT

Yes — the simplified fix removes the **deadlock**, but there are a few input edge cases worth understanding before applying it.

Critical review

| Situation | What happens with the proposed fix | Problem? |
|------------------|------------------------------------|----------|
| First press is A | last_key = None, so A pulls | Correct |

| Situation | What happens with the proposed fix | Problem? |
|--|--|-----------------------------------|
| First press is D | D pulls | Correct |
| A D A D | Every different key pulls | Correct |
| A A A | Only first A pulls | Correct |
| A D while A is still held | D pulls | Correct for this lab |
| Release A after pressing D | Nothing happens | No deadlock |
| Hold A | Initial A pulls; repeated KEYDOWNs are ignored because <code>A == last_key</code> | Safe |
| Hold A with OS key-repeat | Repeated A events don't pull | Safe |
| Press A and D almost simultaneously | Whichever KEYDOWN Pygame processes first pulls; the other pulls if processed afterward | Order-dependent, but not a freeze |
| Press A and D literally simultaneously | Pygame still processes events sequentially, so there is no true "simultaneous" event from our code's perspective | Determined by event order |

One important issue

There's a subtle difference between **"different key"** and **"alternating successful presses."**

Suppose:

A D D A

The behavior is:

A pull
D pull
D ignored
A pull

That's exactly what we want.

But consider:

A D A

while all three keys arrive extremely quickly. All three are processed and pull. Again, that's fine.

Should the first press work for either key?

Yes. Absolutely.

The game shouldn't require the player to start with A specifically.

Because:

```
self.last_key = None
```

the first:

```
A
```

satisfies:

```
A != None
```

and the first:

```
D
```

also satisfies:

```
D != None
```

So **either A or D should be a valid first move.**

Would I change my fix?

For **this particular lab**, I would keep the simplified approach.

The assignment explicitly says:

```
"simply alternating keys (event.key != self.last_key) always pulls"
```

So `last_key` alone is actually a cleaner implementation of the required behavior than trying to maintain a separate lock.

The key invariant becomes:

```
A different key press  pull
Same key press        ignore
```

rather than:

KEYDOWN lock wait for matching KEYUP unlock

The second approach is what created the deadlock in the first place.

One caveat: if Pygame's key-repeat generates repeated KEYDOWN events while holding a key, our `event.key != self.last_key` check already protects against those repeated pulls. So we don't need a separate debounce timer for this assignment.

USER

so please give the updated file content

ASSISTANT

Yes. Since Task 1 only changes `game_engine.py`, you don't need to touch `rope.py`, `player.py`, or `main.py`.

Here is the **complete updated game/game_engine.py**:

```
:::writing{variant="document" id="58321" title="Updated game_engine.py"}
```

```

import random
import pygame
from game.rope import Rope
from game.player import Puller

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.rope = Rope(width, height)
        self.player = Puller(90, height // 2, (50, 120, 220), "PLAYER (A/D)")
        self.computer = Puller(width - 90, height // 2, (220, 80, 50), "COMPUTER")

        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"

        self.computer_pull_cooldown = 180
        self.last_computer_pull = pygame.time.get_ticks()

        self.font_big = pygame.font.SysFont(None, 48)
        self.font_small = pygame.font.SysFont(None, 26)

    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        if event.type == pygame.KEYDOWN:
            if event.key in (pygame.K_a, pygame.K_d):
                if event.key != self.last_key:
                    self.rope.pull_left(1.0)
                    self.last_key = event.key

    def update(self):
        if self.game_state != "PLAYING":
            return

        now = pygame.time.get_ticks()
        if now - self.last_computer_pull >= self.computer_pull_cooldown:
            computer_variance = random.uniform(0.7, 1.2)
            self.rope.pull_right(computer_variance)
            self.last_computer_pull = now

        result = self.rope.check_winner()
        if result:
            self.winner = result
            self.game_state = "GAME_OVER"

    def reset(self):

```

```
self.rope.reset()
self.last_key = None
self.winner = None
self.game_state = "PLAYING"
self.last_computer_pull = pygame.time.get_ticks()
```

```
def render(self, screen):
    screen.fill((30, 32, 36))
```

```
    mud_rect = pygame.Rect(
        self.width // 2 - 120,
        self.height // 2 - 80,
        240,
        160
    )
    pygame.draw.rect(
        screen,
        (45, 38, 30),
        mud_rect,
        border_radius=12
    )
```

```
    self.rope.render(screen)
    self.player.render(screen)
    self.computer.render(screen)
```

```
    inst_surf = self.font_small.render(
        "Alternate [A] and [D] keys rapidly to pull!",
        True,
        (210, 210, 210)
    )
    screen.blit(
        inst_surf,
        (
            self.width // 2 - inst_surf.get_width() // 2,
            40
        )
    )
```

```
    if self.game_state == "GAME_OVER":
        overlay = pygame.Surface(
            (self.width, self.height),
            pygame.SRCALPHA
        )
        overlay.fill((0, 0, 0, 180))
        screen.blit(overlay, (0, 0))
```

```
    win_text = f"{self.winner} WINS!"
    color = (
        80, 220, 80
    )
    if self.winner == "PLAYER"
    else (240, 80, 80)
```

```

)
text_surf = self.font_big.render(
win_text,
True,
color
)
screen.blit(
text_surf,
(
self.width // 2 - text_surf.get_width() // 2,
self.height // 2 - 50
)
)

restart_surf = self.font_small.render(
"Press [R] to Play Again",
True,
(240, 240, 240)
)
screen.blit(
restart_surf,
(
self.width // 2 - restart_surf.get_width() // 2,
self.height // 2 + 10
)
)
)

```

...

What changed

Only three things from the original:

1. Removed:

```
self.is_pull_locked = False
```

2. Removed the KEYUP handling entirely.

3. Player input is now simply:

```

if event.key != self.last_key:
self.ropе.pull_left(1.0)
self.last_key = event.key

```

And reset() sets:

```
self.last_key = None
```

so **either A or D works as the first input after starting/resetting.**

Run it and specifically test **rapid A-D-A-D-A-D while overlapping the key presses**. Once you've confirmed Task 1 works, we can move to **Task 2: dynamic computer difficulty surge**.

USER

Task 2: Implement dynamic computer difficulty surge in `GameEngine.update()`. Right now the computer pulls at a fixed 180 ms interval with random variance. I want it to enter a "panic surge" mode when the center flag (`self.rope.marker_x`) gets pulled close to the player's goal line (`self.rope.left_win_x`): decrease its cooldown and/or increase its pull strength to fight back. Requirements:

- Make the surge threshold, surge cooldown and surge strength named constants or attributes, not magic numbers.
- Surge should turn off again when the flag moves back toward the center.
- Reset everything correctly in `reset()`.
- Optionally show a small on-screen "COMPUTER PANIC!" indicator while surging. Explain your design choice and show only the changed code.

ASSISTANT

Task 2 — Dynamic computer difficulty surge

Design choice

I'd use **both** mechanisms during panic:

- Normal computer cooldown: **180 ms**
- Panic cooldown: **100 ms**
- Normal strength: random **0.7–1.2**
- Panic strength: random **1.0–1.5**

For the trigger, I'll define a **surge threshold as a distance from the player's goal**. For example:

| | | | | |
|-------------|-------|-------|--------|----------|
| PLAYER goal | 80 px | SURGE | CENTER | COMPUTER |
| threshold | | | | |

So if:

```
self.rope.marker_x <= self.rope.left_win_x + self.surge_threshold
```

the computer enters panic mode.

The important part is that this is checked **every frame**, so it automatically turns off when the player pulls the flag back beyond the threshold.

I also recommend storing the state in:

```
self.computer_panic = False
```

rather than trying to infer it elsewhere. `reset()` explicitly resets it to `False`.

Changed code

game/game_engine.py — `__init__()`

Replace the computer difficulty attributes with:

```
self.computer_pull_cooldown = 180
self.computer_surge_cooldown = 100

self.computer_normal_strength_min = 0.7
self.computer_normal_strength_max = 1.2

self.computer_surge_strength_min = 1.0
self.computer_surge_strength_max = 1.5

self.computer_surge_threshold = 80
self.computer_panic = False

self.last_computer_pull = pygame.time.get_ticks()
```

So the important values are now **named attributes**, rather than being buried as magic numbers inside `update()`.

game/game_engine.py — `update()`

Replace only the existing `update()` method with:

```

def update(self):
    if self.game_state != "PLAYING":
        return

    # Check whether the rope is close enough to the player's goal
    surge_limit = (
        self.rope.left_win_x +
        self.computer_surge_threshold
    )

    self.computer_panic = self.rope.marker_x <= surge_limit

    # Use a shorter cooldown and stronger pull during panic
    if self.computer_panic:
        cooldown = self.computer_surge_cooldown
        computer_variance = random.uniform(
            self.computer_surge_strength_min,
            self.computer_surge_strength_max
        )
    else:
        cooldown = self.computer_pull_cooldown
        computer_variance = random.uniform(
            self.computer_normal_strength_min,
            self.computer_normal_strength_max
        )

    now = pygame.time.get_ticks()

    if now - self.last_computer_pull >= cooldown:
        self.rope.pull_right(computer_variance)
        self.last_computer_pull = now

    result = self.rope.check_winner()
    if result:
        self.winner = result
        self.game_state = "GAME_OVER"

```

game/game_engine.py — reset()

Make sure reset() contains:

```

def reset(self):
    self.rope.reset()
    self.last_key = None
    self.winner = None
    self.game_state = "PLAYING"
    self.computer_panic = False
    self.last_computer_pull = pygame.time.get_ticks()

```

The important new line is:

```
self.computer_panic = False
```

Optional indicator

Since you asked for it optionally, I think it's useful for your **after-change video** because you can visibly demonstrate that Task 2 is working.

game/game_engine.py — render()

Add this **after the instruction text is drawn**:

```
if self.computer_panic:
    panic_surf = self.font_small.render(
        "COMPUTER PANIC!",
        True,
        (255, 180, 60)
    )
    screen.blit(
        panic_surf,
        (
            self.width // 2 - panic_surf.get_width() // 2,
            70
        )
    )
```

So the top of the screen will show:

Alternate [A] and [D] keys rapidly to pull!

COMPUTER PANIC!

only while the flag is within the surge zone.

How to test

Start the game and pull rapidly:

A D A D A D ...

Watch the flag.

1. **Far from the left goal:** computer pulls normally.
2. **Flag reaches 80 px from the left goal:** COMPUTER PANIC! appears.
3. Computer should start pulling **more frequently and more strongly**.
4. Pull the flag back toward the center.
5. The indicator should disappear and the computer should return to its normal cooldown/strength.

One thing to notice: **the surge doesn't permanently activate**. `computer_panic` is recalculated every `update()` call, so moving the flag back toward the center automatically switches it off.

USER

Task 3: Implement rope tension and pull animations.

- In `Rope.render()`: replace the static straight line with a rope that shows slight vertical waviness or vibration when tension is high. Tension should be higher when the marker is far from the center. Use `pygame.time.get_ticks()` with `math.sin` and draw the rope as a series of line segments (`pygame.draw.lines`).
- In `Puller.render()`: add a leaning animation so each puller tilts backwards depending on who has pulling momentum (based on marker position relative to the center). Let `render()` accept the needed parameters (for example a lean value from -1 to 1) and update `GameEngine.render()` to pass them. Keep the rope's ends and the flag position visually consistent with `rope.marker_x`. Show only changed code and explain the math simply.

ASSISTANT

Task 3 — Rope tension + puller leaning

I'd keep the physics **visual only** here. We don't need to change how `marker_x` actually moves.

1. Rope waviness — simple math

First calculate how far the flag is from the center:

```
distance = |marker_x - center|
```

Then normalize it to 0 - 1:

```
tension = distance / maximum_possible_distance
```

So:

- flag at center tension ≈ 0
- flag near a goal tension ≈ 1

For the animation, each rope point gets a small vertical offset:

```
y = center_y + amplitude × sin(time + position)
```

`sin()` continuously moves between -1 and +1, creating the vibration/wave effect.

The amplitude is multiplied by tension, so the rope is almost straight at the center and visibly vibrates when pulled far to one side.

Changed code

game/rope.py — imports

Add math:

```
import math
import pygame
```

game/rope.py — render()

Replace the existing render() method with:

```

def render(self, surface):
    center_x = self.screen_width // 2

    # Calculate tension from how far the marker is from the center.
    distance_from_center = abs(self.marker_x - center_x)

    max_distance = max(
        center_x - self.left_win_x,
        self.right_win_x - center_x
    )

    tension = min(distance_from_center / max_distance, 1.0)

    # Stronger tension = larger vertical vibration.
    wave_amplitude = 2 + (10 * tension)

    # Time controls the animation.
    time = pygame.time.get_ticks() / 100.0

    points = []
    rope_start = 60
    rope_end = self.screen_width - 60
    segment_count = 40

    for i in range(segment_count + 1):
        x = rope_start + (
            (rope_end - rope_start) * i / segment_count
        )

        # Keep the rope attached to the flag position.
        # The wave gets stronger as tension increases.
        wave = math.sin(time + i * 0.6) * wave_amplitude

        # Reduce the wave near the ends so the rope stays visually anchored.
        edge_factor = math.sin(math.pi * i / segment_count)
        y = self.center_y + wave * edge_factor

        points.append((int(x), int(y)))

    pygame.draw.lines(
        surface,
        (180, 140, 90),
        False,
        points,
        10
    )

    # Goal markers
    pygame.draw.line(
        surface,
        (50, 200, 50),

```

```

(self.left_win_x, self.center_y - 40),
(self.left_win_x, self.center_y + 40),
4
)

pygame.draw.line(
surface,
(200, 50, 50),
(self.right_win_x, self.center_y - 40),
(self.right_win_x, self.center_y + 40),
4
)

# Center marker
pygame.draw.line(
surface,
(120, 120, 120),
(center_x, self.center_y - 20),
(center_x, self.center_y + 20),
2
)

# Flag remains tied to marker_x.
flag_rect = pygame.Rect(
int(self.marker_x) - 12,
self.center_y - 24,
24,
48
)

pygame.draw.rect(
surface,
(230, 40, 40),
flag_rect,
border_radius=4
)

pygame.draw.rect(
surface,
(255, 255, 255),
flag_rect,
width=2,
border_radius=4
)

```

Important point about the flag

The rope itself is animated around center_y, but the **flag still uses self.marker_x**.

So when the flag moves left/right, its horizontal position remains exactly tied to the actual game state.

The goal lines and center line also remain unchanged.

2. Puller leaning

For the pullers, we'll calculate a lean value between -1 and 1.

Conceptually:

```
marker left of center  PLAYER is pulling  PLAYER leans backward
marker right of center COMPUTER is pulling COMPUTER leans backward
```

We can derive it from the marker position:

```
lean = (marker_x - center_x) / max_distance
```

Then clamp it to [-1, 1].

For the player, we reverse the sign because the player is on the left; for the computer, we use the opposite direction.

game/player.py — render()

Replace the existing render() method with:

```

def render(self, surface, lean=0.0):
    """Draw avatar and label with a pulling lean animation."""

    # Limit lean to the range -1 to 1.
    lean = max(-1.0, min(1.0, lean))

    # Convert lean into a small horizontal displacement.
    lean_offset = int(lean * 12)

    # Body
    body_rect = pygame.Rect(
        self.x - 20 + lean_offset,
        self.y - 35,
        40,
        70
    )

    pygame.draw.rect(
        surface,
        self.color,
        body_rect,
        border_radius=6
    )

    # Head follows the body.
    head_x = self.x + lean_offset
    pygame.draw.circle(
        surface,
        (240, 210, 180),
        (head_x, self.y - 50),
        16
    )

    # Name / control tag stays centered on the player.
    label_surf = self.font.render(
        self.label,
        True,
        (240, 240, 240)
    )

    surface.blit(
        label_surf,
        (
            self.x - label_surf.get_width() // 2,
            self.y + 45
        )
    )

```

This gives the character a visual lean by shifting the body and head horizontally.

game/game_engine.py — render()

Inside render(), replace:

```
self.rope.render(screen)
self.player.render(screen)
self.computer.render(screen)
```

with:

```
center_x = self.width // 2

max_distance = max(
    center_x - self.rope.left_win_x,
    self.rope.right_win_x - center_x
)

position_ratio = (
    self.rope.marker_x - center_x
) / max_distance

position_ratio = max(-1.0, min(1.0, position_ratio))

player_lean = position_ratio
computer_lean = -position_ratio

self.rope.render(screen)
self.player.render(screen, player_lean)
self.computer.render(screen, computer_lean)
```

What this means

If the flag is here:

```
PLAYER          COMPUTER

O ===== | ===== O
FLAG
CENTER
```

both lean very little.

If the player pulls hard:

```
PLAYER          COMPUTER

O ===== FLAG ===== O
PLAYER PULL
```

position_ratio becomes negative.

Therefore:

```
player_lean = position_ratio  
computer_lean = -position_ratio
```

gives the two characters opposite leaning directions.

One limitation of this animation

This is based on **current rope position**, not actual velocity/momentum. So "momentum" here means *which side currently has positional advantage*, rather than physical momentum.

That's appropriate for this task because your Rope currently doesn't maintain velocity—the velocity attribute appears only in reset() and isn't used by the movement methods.

So I **wouldn't introduce actual physics/velocity just for Task 3**. It would unnecessarily change the game's mechanics.

Test

Run the game and check:

1. At the center, the rope should look almost straight.
2. Pull the flag toward the left — rope should become more wavy and the pullers should lean.
3. Pull it back toward the center — waviness should decrease.
4. Push it toward the right via the computer — waviness increases again.
5. The red flag should continue to track rope.marker_x exactly.
6. Make sure the existing win boundaries still work.

This keeps Task 3 **purely visual**—Tasks 1 and 2's gameplay behavior remains intact.

USER

Task 4: Implement a match timer and Sudden Death mode.

- Show an active match timer at the top of the screen (it should not overlap the instruction text).
- If nobody has won after 45 seconds, enter "Sudden Death": double the pull distance of every player keystroke and every computer tick, and show a "SUDDEN DEATH" label.
- The timer must reset on rematch (R), and it should stop when the game is over.
- Make the 45-second value a constant so I can temporarily lower it for testing. Explain how you avoid the timer counting time before the round starts, and show only changed code.

ASSISTANT

Task 4 — Match timer + Sudden Death

Design

I'll use **elapsed time from the moment the round starts**, rather than relying on a continuously running global timer.

The key idea is:

```
reset/start round

match_start_time = pygame.time.get_ticks()

update()

elapsed = current_time - match_start_time

elapsed >= 45 seconds?

Sudden Death
```

This avoids counting time before the round starts because `match_start_time` is set when `GameEngine` is created and **again whenever `reset()` starts a rematch**.

When `game_state` becomes "GAME_OVER", `update()` immediately returns, so the timer stops advancing.

For testing, you can temporarily change:

```
MATCH_DURATION = 45
```

to something like:

```
MATCH_DURATION = 10
```

Changed code

game/game_engine.py — constants

Add these near the imports, before `GameEngine`:

```
MATCH_DURATION = 45
SUDDEN_DEATH_MULTIPLIER = 2
```

`MATCH_DURATION` is in **seconds**.

game/game_engine.py — `__init__()`

Add the timer state after your existing game-state variables:

```
self.match_start_time = pygame.time.get_ticks()
self.sudden_death = False
```

So this section becomes:

```
self.last_key = None
self.winner = None
self.game_state = "PLAYING"

self.match_start_time = pygame.time.get_ticks()
self.sudden_death = False
```

game/game_engine.py — update()

Replace the current update() with:

```

def update(self):
    if self.game_state != "PLAYING":
        return

    now = pygame.time.get_ticks()

    # Calculate how long the current round has been running.
    elapsed_seconds = (now - self.match_start_time) / 1000

    # Enter Sudden Death after the match duration.
    if elapsed_seconds >= MATCH_DURATION:
        self.sudden_death = True

    # Check computer panic state.
    surge_limit = (
        self.ropes.left_win_x +
        self.computer_surge_threshold
    )

    self.computer_panic = self.ropes.marker_x <= surge_limit

    # Select computer cooldown and strength.
    if self.computer_panic:
        cooldown = self.computer_surge_cooldown
        computer_variance = random.uniform(
            self.computer_surge_strength_min,
            self.computer_surge_strength_max
        )
    else:
        cooldown = self.computer_pull_cooldown
        computer_variance = random.uniform(
            self.computer_normal_strength_min,
            self.computer_normal_strength_max
        )

    # Double computer pull distance during Sudden Death.
    if self.sudden_death:
        computer_variance *= SUDDEN_DEATH_MULTIPLIER

    if now - self.last_computer_pull >= cooldown:
        self.ropes.pull_right(computer_variance)
        self.last_computer_pull = now

    result = self.ropes.check_winner()

    if result:
        self.winner = result
        self.game_state = "GAME_OVER"

```

Why multiply computer_variance?

Your `Rope.pull_right()` already interprets its argument as a strength:

```
self.marker_x += int(self.pull_step * strength)
```

THEREFORE

multiplying the strength by 2 automatically doubles the computer's pull distance without modifying `rope.py`.

game/game_engine.py — `handle_event()`

Change the successful player pull from:

```
self.rope.pull_left(1.0)
```

to:

```
pull_strength = (
    SUDDEN_DEATH_MULTIPLIER
    if self.sudden_death
    else 1.0
)

self.rope.pull_left(pull_strength)
```

So the relevant section becomes:

```
if event.type == pygame.KEYDOWN:
    if event.key in (pygame.K_a, pygame.K_d):
        if event.key != self.last_key:
            pull_strength = (
                SUDDEN_DEATH_MULTIPLIER
                if self.sudden_death
                else 1.0
            )

            self.rope.pull_left(pull_strength)
            self.last_key = event.key
```

Thus:

- Normal mode $1.0 \times \text{pull_step} = \mathbf{12\ px}$
- Sudden Death $2.0 \times \text{pull_step} = \mathbf{24\ px}$

game/game_engine.py — `reset()`

Add the timer reset and Sudden Death reset:

```

def reset(self):
    self.rope.reset()
    self.last_key = None
    self.winner = None
    self.game_state = "PLAYING"

    self.computer_panic = False
    self.sudden_death = False
    self.match_start_time = pygame.time.get_ticks()

    self.last_computer_pull = pygame.time.get_ticks()

```

The important part is that `match_start_time` is reset **when R starts a new round**.

game/game_engine.py — render()

You currently display the instruction around `y = 40`, so I'll put the timer at the top-right to prevent overlap.

Add this after the instruction text:

```

elapsed_seconds = 0

if self.game_state == "PLAYING":
    elapsed_seconds = (
        pygame.time.get_ticks() - self.match_start_time
    ) // 1000

elapsed_seconds = min(elapsed_seconds, MATCH_DURATION)

timer_text = f"Time: {elapsed_seconds}s"

timer_surf = self.font_small.render(
    timer_text,
    True,
    (240, 240, 240)
)

screen.blit(
    timer_surf,
    (
        self.width - timer_surf.get_width() - 20,
        15
    )
)

```

Because the timer is at approximately:

Time: 23s

|
Alternate [A] and [D] keys rapidly to pull!

it won't overlap the instruction.

Add the Sudden Death label

Immediately after the timer code, add:

```
if self.sudden_death and self.game_state == "PLAYING":  
    sudden_surf = self.font_small.render(  
        "SUDDEN DEATH",  
        True,  
        (255, 180, 60)  
    )  
  
    screen.blit(  
        sudden_surf,  
        (  
            self.width - sudden_surf.get_width() - 20,  
            45  
        )  
    )
```

So the top-right will look roughly like:

Time: 46s
SUDDEN DEATH

Testing

For your 10-second **after video**, temporarily change:

MATCH_DURATION = 45

to:

MATCH_DURATION = 5

Then you can demonstrate the feature quickly:

1. Start the game.
2. Timer starts at 0s.
3. Play normally for 5 seconds.
4. SUDDEN DEATH appears.

5. Press alternating A/D and observe that each player pull moves the flag **twice as far**.
6. Wait for a computer tick and observe its pull is also doubled.
7. Let someone win.
8. Confirm the timer stops.
9. Press R.
10. Confirm the timer returns to 0s and Sudden Death disappears.

Then restore:

```
MATCH_DURATION = 45
```

before submitting.

One important detail

The timer **doesn't advance inside render()**. `render()` only calculates the current display from `match_start_time`. The actual game-state transition to Sudden Death happens in `update()`.

And because `update()` begins with:

```
if self.game_state != "PLAYING":  
    return
```

once somebody wins, no further Sudden Death logic or computer pulling occurs. That keeps the timer/game state consistent after the match ends.

USER

Please do a final review of the whole project: check that all four tasks work together, that `reset()` restores every new variable (timer, sudden death flag, surge state, key state), and list any bugs or edge cases. Give me a short manual test checklist.

ASSISTANT

Final review

I went through the four changes as one combined system. **The overall design is consistent**, but there are a few things I would fix/check before you record the final video.

1. Task interaction

| Task | Current implementation | Review |
|--------------------|--|----------------------|
| Task 1 — input fix | <code>last_key</code> only; no <code>is_pull_locked</code> | Removes the deadlock |

| Task | Current implementation | Review |
|--------------------------------|---|-------------------------------------|
| Task 2 — panic surge | Flag position determines computer_panic; cooldown/strength change | Turns on/off dynamically |
| Task 3 — rope animation | Tension based on distance from center; sin() animation | Visual only; doesn't affect physics |
| Task 3 — puller lean | Lean based on flag position | Works with player/computer movement |
| Task 4 — timer | match_start_time + elapsed time | Starts per round |
| Task 4 — sudden death | Doubles player/computer pull strength | Works with existing Rope API |
| Reset | Resets game/timer/input/surge state | All new state covered |

Reset check

Your final reset() should contain **all** of these:

```
def reset(self):
    self.rope.reset()
    self.last_key = None
    self.winner = None
    self.game_state = "PLAYING"

    self.computer_panic = False
    self.sudden_death = False
    self.match_start_time = pygame.time.get_ticks()

    self.last_computer_pull = pygame.time.get_ticks()
```

So after pressing R:

- last_key None
- winner None
- game_state "PLAYING"
- computer_panic False
- sudden_death False
- timer starts from the new reset time
- computer cooldown timer starts fresh
- rope marker center via rope.reset()

That's exactly what you want.

Edge cases / things to watch

1. Sudden Death and computer panic can happen simultaneously

This is **intentional**.

If the flag is close to the player's goal *and* 45 seconds have elapsed:

```
Sudden Death
+
Computer Panic

shorter cooldown + stronger pull

then ×2 strength from Sudden Death
```

So the computer can become quite aggressive.

That's not a bug; it's actually a reasonable interaction between Tasks 2 and 4.

2. Sudden Death can technically activate and then immediately end the game

Suppose the timer reaches 45 seconds while the flag is already very close to a goal.

update() sets:

```
self.sudden_death = True
```

and then checks the winner.

If the flag has already crossed the boundary, the game immediately becomes GAME_OVER.

That's fine.

3. Timer display after game over

Your timer code intentionally does:

```
if self.game_state == "PLAYING":
    ...
```

THEREFORE

, once the game ends, it stops calculating new elapsed time.

So the displayed time remains at the last value from the final frame.

That's appropriate for the requirement that the timer stop when the game is over.

4. First key after reset

Because:

```
self.last_key = None
```

both are valid:

```
R  A
```

and:

```
R  D
```

Good.

5. Holding a key

Holding A doesn't repeatedly pull through normal Pygame key-repeat because:

```
event.key != self.last_key
```

becomes false after the first A.

So:

```
A A A A A
```

only produces one pull.

Good.

6. Simultaneous A + D

Pygame processes events sequentially. If both arrive around the same time, whichever KEYDOWN is processed first becomes last_key, and the other different key can also pull.

So you might get:

```
A  pull  
D  pull
```

That's not a deadlock or correctness problem for this assignment.

7. Rope waviness

One thing I'd specifically watch visually.

Your rope wave is generated across the **entire rope**, while the flag is independently drawn at marker_x.

That means the rope's vertical vibration doesn't physically bend around the flag position—it is primarily a visual animation.

That's okay because Task 3 asks for a visual tension effect, not a physics simulation.

The important requirement is satisfied:

```
marker_x  flag horizontal position
```

and marker_x remains the actual game state.

8. Puller "momentum"

The puller lean is technically based on **position**, not actual physical momentum.

That's because the current game has no velocity-based rope physics.

So:

```
flag left of center  player leans  
flag right of center  computer leans
```

This is a visual representation of pulling advantage rather than real momentum.

For this lab, that's perfectly reasonable and avoids unnecessarily rewriting the game's physics.

Short manual test checklist

I'd use this exact checklist before recording your final video:

Basic game

- ☐ Run python main.py
- ☐ Game opens without errors.
- ☐ Rope, flag, player and computer render correctly.

Task 1

- ☐ A D A D repeatedly moves the flag left.
- ☐ Rapid/overlapping A/D presses don't freeze.
- ☐ A A A only pulls once.
- ☐ First key can be either A or D.
- ☐ No is_pull_locked remains in the code.

Task 2

- ☐ Pull flag close to player's goal.

- ☐ COMPUTER PANIC! appears.
- ☐ Computer becomes noticeably more aggressive.
- ☐ Pull flag back toward center.
- ☐ Panic indicator disappears.

Task 3

- ☐ Rope is mostly straight near center.
- ☐ Rope visibly vibrates/waves when far from center.
- ☐ Player/computer lean according to which side has the advantage.
- ☐ Flag still moves exactly with marker_x.

Task 4

Temporarily use:

```
MATCH_DURATION = 5
```

- ☐ Timer starts at approximately 0s.
- ☐ Timer counts upward.
- ☐ After 5 seconds, SUDDEN DEATH appears.
- ☐ Player pulls move approximately twice as far.
- ☐ Computer pulls approximately twice as far.
- ☐ Game can still end normally.

Reset

- ☐ Let the game reach Game Over.
- ☐ Timer stops.
- ☐ Press R.
- ☐ Flag returns to center.
- ☐ Timer returns to 0s.
- ☐ SUDDEN DEATH disappears.
- ☐ COMPUTER PANIC! disappears.
- ☐ First A **or** D works immediately.
- ☐ Computer cooldown starts fresh.

Before submission

- ☐ Change MATCH_DURATION back to 45.
- ☐ Remove any temporary debugging prints.
- ☐ Run the game one final time.
- ☐ Record the **before** video using the original code.
- ☐ Record the **after** video with all four tasks.
- ☐ Push only your updated code to your personal repo.

- ☐ Export/save the ChatGPT conversation for the required chat-history deliverable.

One final code-quality check: make sure you haven't accidentally left the old `self.is_pull_locked` initialization or `KEYUP` logic in `game_engine.py`. If it is still there, remove it; Task 1 is supposed to use `last_key` alone.